五子棋思考过程

一、整体思路

将问题拆分为两个板块：

1. **游戏逻辑**：棋盘数据结构、落子规则、胜负判定、平局判定。
2. **控制台交互**：绘制棋盘、读取鼠标事件、坐标转换、光标控制。

两者通过 `main.c` 协调，逻辑与界面分离。

二、字符坐标与棋盘坐标的换算

棋盘采用如下样式，每格宽 4 字符、高 2 字符：

```
+---+---+---+---+---+---+---+---+---+ | . | . | . | . | . | . | . | . | . | +---+---+---+---+---+---+---+---+---+
```

设棋盘左上角为 `(board_left, board_top)`：

- 第 r 行分隔线 y 坐标：`board_top + 2*r`
- 第 r 行棋子 y 坐标：`board_top + 2*r + 1`
- 第 c 列棋子 x 坐标：`board_left + 4*c + 2`

反过来，从鼠标坐标 `(mx, my)` 推算棋盘坐标：

- `r = (my - board_top - 1) / 2`
- `c = (mx - board_left - 2) / 4`

三、为什么要设计两种坐标?

- **字符坐标**：终端屏幕上的物理位置，用于绘制和鼠标事件。
- **棋盘坐标**：逻辑上的行列索引，用于游戏规则判断。

两种坐标分离后，绘制和游戏逻辑互不干扰。如果以后改变棋盘样式（比如每格 6 字符宽），只需修改 `console.c` 中的换算公式，`game.c` 完全不用改。

四、胜负判定

每次落子后，只需检查**刚落子位置**在四个方向上是否形成五连。

四个方向：

方向	dr	dc
横	0	1
纵	1	0
主对角线	1	1
副对角线	1	-1

对每个方向，从落子点向正、反两个方向各数 4 步，统计连续同色棋子数。任意方向 ≥5 即判胜。

这种“以落子点为中心”的判定只需 32 次比较，效率极高。

五、平局判定

用 `move_count` 记录落子总数，当 `move_count == 81` 且无人获胜时判平局。

六、鼠标输入处理

使用 Windows Console API：

1. `SetConsoleMode` 开启 `ENABLE_MOUSE_INPUT`，关闭 `ENABLE_QUICK_EDIT_MODE`。
2. `ReadConsoleInput` 循环读取事件，过滤出 `MOUSE_EVENT`。
3. 检查 `dwButtonState & FROM_LEFT_1ST_BUTTON_PRESSED` 判断左键。
4. 按下后等待左键松开，防止一次点击触发多次。

七、状态栏位置

状态栏放在棋盘下方，因为：

- 不遮挡棋盘，视觉上更清晰。
- 棋盘本身占 19 行（9 行棋子 + 10 行分隔线），下方有足够空间。
- 与原生 `ls` 等工具的输出习惯一致。

八、注意事项

- 必须在真正的 Windows 终端（cmd）中运行，VS Code 内置终端会拦截鼠标事件。
- 程序退出前恢复控制台模式和光标可见性。
- 无效点击（越界、已占用）直接忽略，不改变游戏状态。
- 中文乱码时可在终端执行 `chcp 65001`。

九、测试要点

- 点击棋盘外：不应落子。
- 点击已占用格子：不应落子。
- 斜向五连：应正常判胜。
- 棋盘填满：应判平局。

代码部分

game.h

#ifndef GAME_H #define GAME_H

#include <stdbool.h>

#define BOARD_SIZE 9 #define WIN_LENGTH 5 #define EMPTY_CELL '.'

typedef enum { GAME_RUNNING, GAME_X_WINS, GAME_O_WINS, GAME_DRAW } GameStatus;

typedef enum { MOVE_ACCEPTED, MOVE_OUT_OF_BOUNDS, MOVE_OCCUPIED } MoveResult;

typedef struct { char cells[BOARD_SIZE][BOARD_SIZE]; char current_player; int move_count; GameStatus status; } Game;

/* 初始化游戏 */ void game_init(Game *game);

/* 在 (row, col) 处落子，自动切换玩家并检查胜负 */ MoveResult game_place(Game *game, int row, int col);

/* 检查 (row, col) 处是否形成五连 */ bool game_check_winner(Game *game, int row, int col);

#endif

game.c

#include "game.h"

void game_init(Game *game) { for (int r = 0; r < BOARD_SIZE; r++) { for (int c = 0; c < BOARD_SIZE; c++) { game->cells[r][c] = EMPTY_CELL; } } game->current_player = 'X'; game->move_count = 0; game->status = GAME_RUNNING; }

bool game_check_winner(Game *game, int row, int col) { char player = game->cells[row][col];

```
// 四个方向：横、纵、主对角线、副对角线
int dr[4] = {0, 1, 1, 1};
int dc[4] = {1, 0, 1, -1};

for (int d = 0; d < 4; d++) {
    int count = 1;

    // 正方向
    for (int i = 1; i < WIN_LENGTH; i++) {
        int r = row + dr[d] * i;
        int c = col + dc[d] * i;
        if (r < 0 || r >= BOARD_SIZE || c < 0 || c >= BOARD_SIZE) break;
        if (game->cells[r][c] != player) break;
        count++;
    }

    // 反方向
    for (int i = 1; i < WIN_LENGTH; i++) {
        int r = row - dr[d] * i;
        int c = col - dc[d] * i;
        if (r < 0 || r >= BOARD_SIZE || c < 0 || c >= BOARD_SIZE) break;
        if (game->cells[r][c] != player) break;
        count++;
    }

    if (count >= WIN_LENGTH) return true;
```

```
    }
    return false;
```

}

MoveResult game_place(Game *game, int row, int col) { if (row < 0 || row >= BOARD_SIZE || col < 0 || col >= BOARD_SIZE) { return MOVE_OUT_OF_BOUNDS; } if (game->cells[row][col] != EMPTY_CELL) { return MOVE_OCCUPIED; }

```
    game->cells[row][col] = game->current_player;
    game->move_count++;

    if (game_check_winner(game, row, col)) {
        game->status = (game->current_player == 'X') ? GAME_X_WINS : GAME_O_WINS;
    } else if (game->move_count == BOARD_SIZE * BOARD_SIZE) {
        game->status = GAME_DRAW;
    } else {
        game->current_player = (game->current_player == 'X') ? 'O' : 'X';
    }

    return MOVE_ACCEPTED;
```

}

console.h

#ifndef CONSOLE_H #define CONSOLE_H

#include "game.h"

typedef struct { int board_top; // 棋盘第一行（+---+）在终端中的 Y 坐标 int board_left; // 棋盘左边缘在终端中的 X 坐标 int cell_width; // 每个格子占多少个字符宽（4） int cell_height; // 每个格子占多少个字符高（2） } GameConsole;

void console_init(GameConsole *gc); void console_cleanup(void); void console_draw(Game *game, GameConsole *gc); void console_wait_click(GameConsole *gc, int *row, int *col);

#endif

console.c

#include <stdio.h> #include <windows.h> #include "console.h"

static HANDLE hStdin; static HANDLE hStdout; static DWORD original_mode;

static void gotoxy(int x, int y) { COORD coord; coord.X = (SHORT)x; coord.Y = (SHORT)y; SetConsoleCursorPosition(hStdout, coord); }

void console_init(GameConsole *gc) { hStdin = GetStdHandle(STD_INPUT_HANDLE); hStdout = GetStdHandle(STD_OUTPUT_HANDLE);

```
// 保存原模式，开启鼠标输入，关闭快速编辑模式
GetConsoleMode(hStdin, &original_mode);
DWORD mode = original_mode | ENABLE_MOUSE_INPUT | ENABLE_EXTENDED_FLAGS;
mode &= ~ENABLE_QUICK_EDIT_MODE;
SetConsoleMode(hStdin, mode);

// 隐藏光标
CONSOLE_CURSOR_INFO info;
GetConsoleCursorInfo(hStdout, &info);
info.bVisible = FALSE;
SetConsoleCursorInfo(hStdout, &info);

system("cls");

gc->board_top   = 1;
gc->board_left  = 0;
gc->cell_width  = 4;
gc->cell_height = 2;
```

}

void console_cleanup(void) { SetConsoleMode(hStdin, original_mode); CONSOLE_CURSOR_INFO info; GetConsoleCursorInfo(hStdout, &info); info.bVisible = TRUE; SetConsoleCursorInfo(hStdout, &info); }

void console_draw(Game *game, GameConsole *gc) { // 棋盘 for (int r = 0; r < BOARD_SIZE; r++) { // 分隔线 gotoxy(gc->board_left, gc->board_top + r * gc->cell_height); for (int c = 0; c < BOARD_SIZE; c++) { printf("+---"); } printf("+");

```
    // 棋子行
    gotoxy(gc->board_left, gc->board_top + r * gc->cell_height + 1);
    for (int c = 0; c < BOARD_SIZE; c++) {
        printf("| %c ", game->cells[r][c]);
    }
    printf("|");
}

// 最后一条分隔线
gotoxy(gc->board_left, gc->board_top + BOARD_SIZE * gc->cell_height);
for (int c = 0; c < BOARD_SIZE; c++) {
    printf("+---");
}
printf("+");

// 状态栏
gotoxy(gc->board_left, gc->board_top + BOARD_SIZE * gc->cell_height + 2);
switch (game->status) {
    case GAME_RUNNING:
```

```
        printf("当前玩家: %c   步数: %d   ", game->current_player, game-
>move_count);
        break;
    case GAME_X_WINS:
        printf("X 获胜! 总步数: %d          ", game->move_count);
        break;
    case GAME_O_WINS:
        printf("O 获胜! 总步数: %d          ", game->move_count);
        break;
    case GAME_DRAW:
        printf("平局! 总步数: %d            ", game->move_count);
        break;
}
fflush(stdout);
```

}

void console_wait_click(GameConsole *gc, int *row, int *col) { INPUT_RECORD record; DWORD events;

```
while (1) {
    ReadConsoleInput(hStdin, &record, 1, &events);
    if (record.EventType != MOUSE_EVENT) continue;

    MOUSE_EVENT_RECORD mer = record.Event.MouseEvent;
    if (!(mer.dwButtonState & FROM_LEFT_1ST_BUTTON_PRESSED)) continue;

    int mx = mer.dwMousePosition.X;
    int my = mer.dwMousePosition.Y;

    // 坐标转换
    int c = (mx - gc->board_left - 2) / gc->cell_width;
    int r = (my - gc->board_top - 1) / gc->cell_height;

    // 等待左键松开，避免重复触发
    while (1) {
        ReadConsoleInput(hStdin, &record, 1, &events);
        if (record.EventType == MOUSE_EVENT) {
            MOUSE_EVENT_RECORD mer2 = record.Event.MouseEvent;
            if (!(mer2.dwButtonState & FROM_LEFT_1ST_BUTTON_PRESSED)) break;
        }
    }

    *row = r;
    *col = c;
    return;
}
```

}

main.c

#include <stdio.h> #include <conio.h> #include "game.h" #include "console.h"

int main(void) { Game game; GameConsole gc;

```
    game_init(&game);
    console_init(&gc);

    while (game.status == GAME_RUNNING) {
        console_draw(&game, &gc);

        int row, col;
        console_wait_click(&gc, &row, &col);

        // 无效点击直接忽略，游戏状态不变
        game_place(&game, row, col);
    }

    console_draw(&game, &gc);

    printf("\n\n按任意键退出...");
    _getch();

    console_cleanup();
    return 0;
```

}